



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Sprint 2

Grupo:

Jorge Carnicero Príncipe,

Andrés Gil Vicente,

Jorge González Pérez

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

1. Introducción.....	3
1.1. Contexto.....	3
1.2. Descripción del problema y objetivos de este sprint.....	3
1.3. Justificación.....	3
2. Metodología.....	4
2.1. Descripción del entorno de desarrollo.....	4
2.2. Diseño de la solución.....	4
2.3. Pruebas realizadas.....	4
3. Resultados.....	5
3.1. Descripción de los resultados.....	5
3.2. Pantallazos de la ejecución.....	5
3.3. Discusión de los resultados.....	6
4. Conclusión.....	7
4.1. Resumen del proceso.....	7
4.2. Principales logros.....	7

1. Introducción

1.1. Contexto

En el marco del proyecto TradeData, tras completar con éxito el Sprint 1 donde se estableció la capa de almacenamiento de datos históricos en Amazon S3, el Sprint 2 se centra en implementar el gobierno de los datos mediante AWS Glue Data Catalog y AWS Glue Crawler.

1.2. Descripción del problema y objetivos de este sprint

El objetivo principal de este Sprint 2 es catalogar automáticamente los metadatos de los datos históricos almacenados en S3 durante el Sprint 1, permitiendo así:

- Establecer un gobierno centralizado de los datos mediante el AWS Glue Data Catalog.
- Automatizar el descubrimiento del esquema de las tablas mediante AWS Glue Crawler.
- Preparar la infraestructura para consultas SQL y análisis posteriores mediante servicios como AWS Athena o AWS Glue ETL.
- Mantener un inventario actualizado de los datasets disponibles con sus metadatos correspondientes (columnas, tipos de datos, particiones).

1.3. Justificación

Este Sprint 2 es fundamental porque establece la capa de metadatos del Data Lake, transformando un repositorio de archivos en S3 en un catálogo estructurado y consultable. Mediante Glue Data Catalog facilitamos la trazabilidad, el descubrimiento de datos y la integración con herramientas de análisis. La automatización completa mediante scripting Python garantiza la reproducibilidad y escalabilidad del sistema.

2. Metodología

A continuación, se explica detalladamente cómo se ha diseñado e implementado la solución al objetivo planteado para el sprint, así como las principales decisiones técnicas tomadas y las tecnologías empleadas.

2.1. Descripción del entorno de desarrollo

- **Lenguaje y Librerías:** Se ha utilizado **Python 3** junto con la librería de **Boto3** para interactuar programáticamente con los servicios de AWS.
- **Servicios Cloud:**
 - **AWS Glue:** Para la creación de la Base de Datos de metadatos y el Crawler.
 - **IAM:** Gestión de permisos mediante roles (`GLUE_ROLE_ARN`).
 - **S3:** Fuente de datos (`trade-data-big-daddyks-trading`).
- **Entorno de Ejecución:** El script está diseñado para ejecutarse localmente o en un entorno de CI/CD, configurando las credenciales y la región (`eu-south-2`) a través de variables de entorno para mayor seguridad.

2.2. Diseño de la solución

La arquitectura se basa en un enfoque de **Infraestructura como Código (IaC)** imperativo. El script `glue_creations.py` implementa una lógica idempotente:

1. **Verificación de Existencia:** Antes de crear recursos, comprueba si ya existen en la cuenta de AWS.
2. **Creación/Actualización:** Si la base de datos `trade_data_imat3a05` no existe, la crea. Si el Crawler ya existe, lo actualiza (`update_crawler`) para reflejar cambios en el código o permisos; si no, lo crea desde cero.
3. **Ejecución:** Lanza el Crawler automáticamente para indexar los datos particionados (`Asset/Year/Month`) almacenados en S3.

Diagrama de flujo lógico: Inicio -> Leer Variables Env -> ¿Existe DB? (No: Crear) -> ¿Existe Crawler? (Sí: Actualizar / No: Crear) -> Iniciar Crawler -> Fin

2.3. Pruebas realizadas

Se realizaron pruebas de ejecución incremental para verificar la robustez del script:

1. **Ejecución en limpio:** Se validó que el script creara todos los recursos desde cero, cuando ninguno de estos existía previamente.
2. **Re-ejecución:** Se validó que, al volver a ejecutar el script sobre una infraestructura ya desplegada, no se produjeran errores y se actualizaran las configuraciones correctamente.
3. **Verificación de resultados:** Se comprobó manualmente en la consola de AWS que las tablas generadas coincidieran con el esquema de los CSV.

3. Resultados

En esta sección se presentan las evidencias de la ejecución del script y la validación de la infraestructura generada.

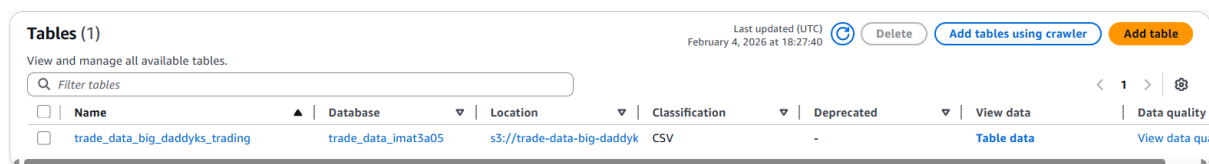
3.1. Descripción de los resultados

La ejecución del script `glue_creations.py` resultó exitosa. Se creó correctamente la base de datos `trade_data_imat3a05` en la región `eu-south-2` y se configuró el Crawler. Tras su finalización, el Crawler catalogó los datos de S3, generando una tabla con el esquema correcto (`datetime`, `open`, `high`, `low`, `close`, `volume`, `asset`, `year`, `month`) y reconociendo las particiones temporales.

3.2. Pantallazos de la ejecución

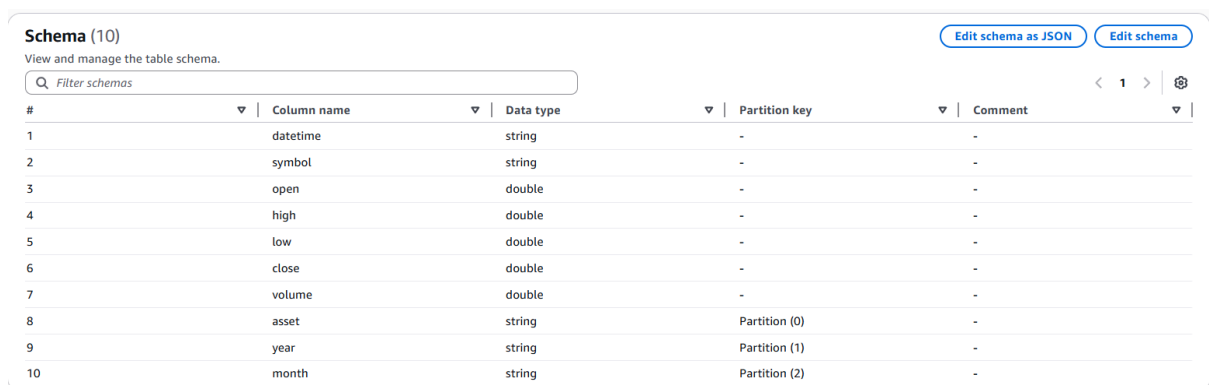
```
(tbd_env) PS C:\Users\User\OneDrive - Universidad Pontificia Comillas\ICAI\3º\2. CUATRI\TBD\PRACTICA\BigDadiks2> python .\glue_creations.py
[GLUE] Iniciando gestión en Región: eu-south-2
Using Role: Crawler_Role
[GLUE] DB 'trade_data_imat3a05' ya existe.
[GLUE] Creando Crawler...
[GLUE] Crawler 'crawler_imat3a05_trading' lanzado con éxito.
(tbd_env) PS C:\Users\User\OneDrive - Universidad Pontificia Comillas\ICAI\3º\2. CUATRI\TBD\PRACTICA\BigDadiks2>
```

Figura 1: Ejecución del script en terminal mostrando la conexión y creación de recursos.



Name	Database	Location	Classification	Deprecated	View data	Data quality
trade_data_big_daddyks_trading	trade_data_imat3a05	s3://trade-data-big-daddyk	CSV	-	Table data	View data qu

Figura 2: AWS Glue mostrando la Base de Datos.



#	Column name	Data type	Partition key	Comment
1	datetime	string	-	-
2	symbol	string	-	-
3	open	double	-	-
4	high	double	-	-
5	low	double	-	-
6	close	double	-	-
7	volume	double	-	-
8	asset	string	Partition (0)	-
9	year	string	Partition (1)	-
10	month	string	Partition (2)	-

Partitions (49)
The list of partitions for this table.

Filter partitions

	asset	year	month	Files	Properties
☐	SOLUSD	2022	09	View files	View Properties
☐	SOLUSD	2026	01	View files	View Properties
☐	SOLUSD	2024	06	View files	View Properties
☐	SOLUSD	2022	08	View files	View Properties
☐	SOLUSD	2023	09	View files	View Properties
☐	SOLUSD	2024	09	View files	View Properties
☐	SOLUSD	2025	03	View files	View Properties
☐	SOLUSD	2025	08	View files	View Properties
☐	SOLUSD	2024	04	View files	View Properties
☐	SOLUSD	2025	06	View files	View Properties
☐	SOLUSD	2023	08	View files	View Properties
☐	SOLUSD	2023	01	View files	View Properties
☐	SOLUSD	2025	10	View files	View Properties
☐	SOLUSD	2022	03	View files	View Properties
☐	SOLUSD	2024	01	View files	View Properties
☐	SOLUSD	2023	02	View files	View Properties
☐	SOLUSD	2025	09	View files	View Properties
☐	SOLUSD	2023	03	View files	View Properties
☐	SOLUSD	2024	07	View files	View Properties
☐	SOLUSD	2022	05	View files	View Properties

Figura 3: Información dentro de la tabla obtenida por el crawler

3.3. Discusión de los resultados

- **Comportamiento del sistema:** El sistema se comportó de la manera prevista. El script detectó correctamente el estado de los recursos y actuó en consecuencia (creando o actualizando).
- **Factores influyentes:** La correcta creación del rol de de IAM del crawler, permitiendo únicamente que lea en S3 y un acceso total a AWSGlueConsole

Crawler_Role Info
Allows Glue to call AWS services on your behalf.

Delete

Summary Edit

Creation date
January 29, 2026, 19:51 (UTC+01:00)

ARN
arn:aws:iam::256876712551:role/Crawler_Role

Last activity
5 days ago

Maximum session duration
1 hour

Permissions Trust relationships Tags Last Accessed Revoke sessions

Permissions policies (2) Info
You can attach up to 10 managed policies.

Simulate Remove Add permissions

Search Filter by Type All types

Policy name	Type	Attached entities
☐ AmazonS3ReadOnlyAccess	AWS managed	1
☐ AWSGlueConsoleFullAccess	AWS managed	2

Figura 4: Rol creado para el Crawler con permiso de Lectura en S3 y Total sobre AWS Glue

- **Comparación con lo esperado:** Se esperaba que Glue identificara las columnas del CSV automáticamente. El resultado confirmó esto, aunque se observó que el Crawler infiere la columna datetime como string.

4. Conclusión

4.1. Resumen del proceso

Se ha completado la historia de usuario, pasando por el desarrollo de la lógica de automatización en Python, hasta la validación final en la nube de AWS. Transformando un conjunto de archivos CSV aislados en un activo de datos catalogado y consultable.

4.2. Principales logros

1. **Automatización:** Con ejecutar el archivo.py se hace automáticamente, buscaremos en pasos futuros crear un workflow y con jobs de glue si es posible.
2. **Escalabilidad:** La implementación del Crawler asegura que la arquitectura soportará la carga de nuevos datos históricos en futuros sprints sin requerir cambios en el código.
3. **Catalogación automática** de esquemas tipos de datos y particiones de los datos históricos en S3.
4. **Preparación de la infraestructura para consultas SQL** (Athena), procesamiento Spark (Glue ETL) y análisis en sprints futuros.